DiaMap: A Differentiable LUT Mapper for Area Optimization

Zhenxuan Xie
The Chinese University of Hong Kong
zxxie24@cse.cuhk.edu.hk

Evangeline F.Y. Young
The Chinese University of Hong Kong
fyyoung@cse.cuhk.edu.hk

Abstract

Technology mapping is a pivotal step in logic synthesis, where area optimization remains a critical yet NP-hard challenge. State-of-the-art cut-based methods, such as ABC's IfMap, rely on greedy dynamic programming heuristics that are susceptible to local optima. To address the limitation, we propose DiaMap, the first GPU-accelerated, differentiable, area-oriented LUT mapping framework designed for the concurrent optimization of millions of nodes. Our approach introduces a diverse initial cut enumeration strategy that incorporates fanout-aware area flow and unseen leaves to broaden the search space. We relax the discrete cut selection into a continuous, GPU-friendly formulation and theoretically prove that its optimal value coincides with the minimum achievable mapped circuit area. Finally, a fast top-p cut reselection mechanism is utilized to rectify minor structural errors and further refine the area. Experiments on arithmetic circuits from the EPFL benchmark suite show that DiaMap achieves average reductions of 14.91% in area and 19.45% in delay, along with a 2× average runtime speedup over the area-oriented IfMap.

1 Introduction

Look-up table (LUT) mapping is a fundamental step in logic synthesis, where a circuit represented in a technology independent form, such as an And-Inverter Graph (AIG), is transformed into a network of k-input LUTs for FPGA implementation. Owing to their flexibility and efficiency, LUT mapping algorithms have been widely adopted in modern synthesis flows. Beyond FPGA design [16], LUT mapping also serves as a core building block in logic optimization. In particular, it is often used as a technology-independent optimization framework, where costs are evaluated abstractly [2, 28]. This abstraction enables structural exploration without binding to a specific library, making it applicable to both FPGA and ASIC design flows.

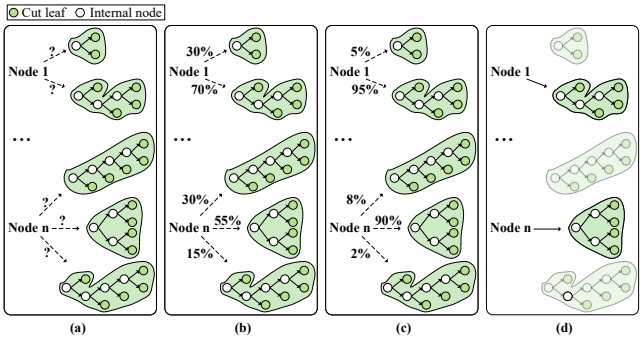


Figure 1: Illustration of differentiable mapping. (a) Cut enumeration. (b) Continuous relaxation of cur selection. (c) Optimization of the probabilities in a differentiable manner. (d) Inducing the final cuts from the optimized probabilities.

Among various optimization metrics [14, 26], circuit depth and area are the primary objectives in LUT mapping. The circuit depth directly influences signal delay, while a reduced area footprint translates to savings in both spatial and power costs. Specifically, the LUT-based technology mapping problem [18, 19] can be formulated as a graph clustering or covering task [24]. While optimizing for circuit depth reduction is computationally tractable in polynomial time (e.g., via FlowMap [5]), area optimization remains NP-hard [17].

To address area optimization in increasingly complex circuits, various cut-based LUT mapping methods have been proposed [3, 4, 6, 23]. These methods employ dynamic programming to enumerate k-feasible cuts for each node and select one based on metrics such as delay and area flow. Among them, IfMap in ABC [3] is widely used. It iteratively alternates between cut enumeration and selection using delay and area as guidance. Despite its effectiveness, the greedy nature of IfMap makes it prone to local optima [23] and it also suffers from slow runtime on large designs.

Recently, numerous GPU-accelerated algorithms have demonstrated substantial efficiency improvements across various EDA problems [8, 12, 13, 25, 27]. Concurrently, differentiable programming has shown that subgradient optimization from

This work is licensed under a Creative Commons Attribution 4.0 International License.
ICCAD '26, San Jose, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2873-0/2026/11
https://doi.org/10.1145/3831252.3833948

a global perspective can yield superior results, as evidenced by tools in placement [11], routing [10], and gate sizing [7].

Notably, GradMap [22] proposed a gradient-based technology mapper for ASICs that achieves certain metric-driven improvements including area. However, it fails to fully leverage the advantages of differentiable optimization due to several fundamental limitations. First, it still heavily relies on ABC's cut enumeration. Second, its area loss formulation employs a `min` operation, which not only causes gradient truncation but also introduces significant computational difficulties. Furthermore, there exists a non-negligible gap between this proxy loss and the true area. Compounded by this complex and GPU-unfriendly cost function, GradMap requires over 20,000 times the runtime of ABC's mapper.

In this paper, we propose DiaMap, an area-oriented LUT mapping framework that differentiates the cut selection procedure (as shown in Figure 1). Our method employs a diversified approach to initial cut enumeration and tackles the discrete cut selection problem by differentiating the probability of each cut. Our primary contributions are summarized as follows:

- We propose DiaMap, a GPU-accelerated, differentiable, area-oriented LUT mapping framework designed for the concurrent area optimization of millions of nodes. To the best of our knowledge, this is the first GPU-accelerated differentiable LUT mapper.
- We introduce a diverse initial cut enumeration strategy that incorporates fanout-aware area flow and unseen leaves, providing a more comprehensive global view of the mapping choices.
- We formulate a continuous, GPU-friendly optimization for differentiable cut selection that scales efficiently to large designs, and theoretically prove that its optimal value equals the minimum mapped circuit area.
- To rectify minor structural errors introduced by pure differentiable mapping, we present a fast top-p reselection mechanism for post-mapping area improvement.

We conducted extensive experiments on the EPFL benchmark suite [1] to evaluate our algorithm. Compared to the area-oriented IfMap implemented in the academic logic synthesis tool ABC [3], DiaMap achieves an average reductions of 14.91% in area and 19.45% in delay on the arithmetic cases. Furthermore, DiaMap achieves an average runtime speedup of 2× over IfMap.

The remainder of this paper is organized as follows. Section 2 introduces the background of LUT mapping and reviews ABC's IfMap. Section 3 describes our proposed LUT mapping methodology. Section 4 presents the experimental results, and Section 5 concludes the paper.

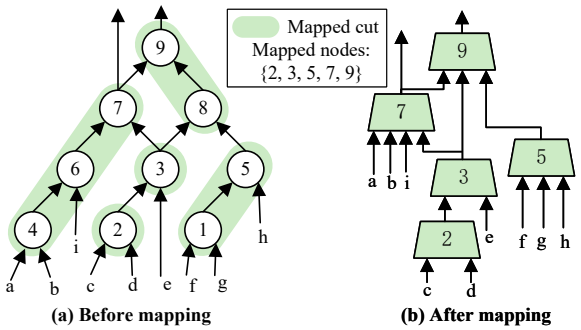


Figure 2: Illustration of LUT mapping

2 Preliminaries

2.1 Background

A Boolean network models a combinational circuit as a directed acyclic graph (DAG), where each node represents a Boolean function and edges capture signal dependencies among nodes. For any node, its transitive fanins and fanouts correspond to all upstream and downstream nodes in the graph, respectively. Immediate predecessors and successors are referred to as fanins and fanouts. The primary inputs (PIs) and primary outputs (POs) define the boundary of the network, acting as its sources and sinks.

An And-Inverter Graph (AIG) is a restricted form of Boolean network in which all nodes are two-input AND gates, with optional inversion on their inputs. In contrast, a k-LUT network allows each node to implement any Boolean function of up to k inputs. For both AIGs and LUT networks, the delay of a node is defined as the length of the longest path from any PI to that node, which can be computed recursively as

$$d_N(n) = 1 + \max_{n' \in FI(n)} d_N(n'), \tag{1}$$

where $FI(n)$ denotes the set of fanins of node n, and the constant 1 represents the intrinsic delay of the corresponding logic element. The delay of the entire network is given by the maximum delay among all POs. For LUT networks, area is measured by the total number of LUT nodes.

For a node n, a cut is defined as a set of nodes that intersects every path from any PI to n. The logic cone induced by a cut consists of the nodes that lie in both the transitive fanin region of n and the transitive fanout regions of the cut nodes, including n itself.

LUT mapping converts a Boolean network into an LUT-based implementation. Although this work considers AIGs as the input representation, the proposed method can be extended to other network types. The mapping process determines a final cut for each node and selects a subset of nodes such that the corresponding logic cones collectively

cover all internal (non-PI) nodes. An example of LUT mapping with $k = 4$ is illustrated in Fig. 2.

2.2 Review of IfMap

Implemented in ABC, IfMap features two primary mapping modes: delay-driven and area-driven. The former prioritizes minimizing delay and subsequently optimizes area under delay constraints, while the latter focuses solely on area minimization without considering delay constraints.

For each node n, IfMap maintains a cut set $\phi(n)$ storing up to C k-feasible cuts. The final cut, denoted $F^*(n)$, is selected for LUT mapping. The algorithm travels the node in a topological order, and the cut sets of two fanins are merged to enumerate a candidate subset $\Phi_m(n)$. In each iteration, an evaluation criterion (e.g., area flow) sorts $\Phi_m(n)$, retaining only the top C cuts in $\phi(n)$, and the highest-ranked cut is selected as the final cut $F^*(n)$.

Let LUT(n) represent the LUT constructed by the final cut of n, The reference number $R(n)$ represents the fanout count of LUT(n). When LUT(n) is removed, the reference numbers of LUT(n)'s leaves are decremented by one, and any leaves whose reference counts drop to zero are recursively removed. Conversely, constructing LUT(n) performs the inverse operation: the reference counts of its leaf nodes are incremented, and any node whose reference count increases from zero to one is recursively activated.

There are two metrics used in IfMap for area optimization:

2.2.1 Area Flow. For a cut X, its area flow is defined as:

$$AF(X) = \sum_{n \in X} \frac{AF(n)}{R(n)} + 1 \tag{2}$$

Here, n is leaf of cut X. $AF(n) = 0$ for PIs and $AF(n) = AF(F^*(n))$ for all other nodes. Before the first iteration, $R(n)$ is initialized to the fanout count of node n in the AIG. It is subsequently updated after each mapping iteration based on the LUT fanout count.

2.2.2 Exact Area. When we construct a new cut X, its exact area $EA(X)$ is defined as:

$$EA(X) = \sum_{n \in X} EA(n) \cdot I(n) + 1 \tag{3}$$

Similarly, $EA(n) = 0$ for PIs and $EA(n) = EA(F^*(n))$ for others. The indicator function $I(n) = 1$ if $R(n) = 0$, and $I(n) = 0$ otherwise. This metric is typically applied in topological order to enable incremental evaluation. Specifically, given a current mapping result, we first remove $F^*(n)$, it then evaluates all candidate cuts individually according to Eq. (3), and finally selects the cut that yields the minimum exact area.

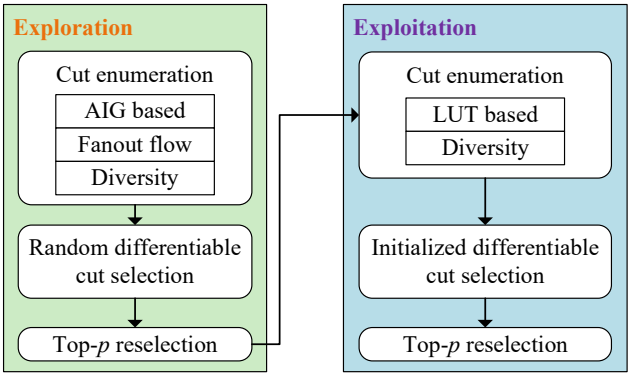


Figure 3: Overview of DiaMap

3 Proposed Algorithm: DiaMap

This section introduces our differentiable LUT mapper, DiaMap. We begin with an overview of the mapping framework, followed by a description of how diverse candidate cuts are enumerated by adapting fanout-aware area flow and unseen leaves. Next, we present a continuous, GPU-friendly objective for differentiable cut selection which is efficient for large designs. Finally, we introduce a top-p selection strategy to further optimize the area.

3.1 Main Framework

The overall framework of DiaMap is illustrated in Figure 3. The mapping process consists of two stages: exploration and exploitation. Both stages encompass three phases: cut enumeration, differentiable cut selection, and top-p reselection.

Cut Enumeration: This phase is foundational for cut-based mapping. For each node n, a maximum of C priority k-feasible cuts are computed. During the exploration stage, we adopt three strategies to generate a diverse set of cuts. In the exploitation stage, two strategies are employed to encourage convergence while still preserving diversity.

Differentiable Cut Selection: In this phase, a final cut is selected for each node by optimizing the probabilities of candidate cuts within a differentiable framework. The exploration stage is initialized with a uniform distribution over cuts, while the exploitation stage is initialized from the exploration results. The final cut is then selected based on the optimized probabilities.

Top-p Reselection: During this phase, we perform a cut reselection by comparing the currently final cuts with the top-p highest-probability cuts. This allows the algorithm to precisely evaluate the area impact and repair some errors of the differentiable cut selection. The parameter p serves to balance quality and efficiency.

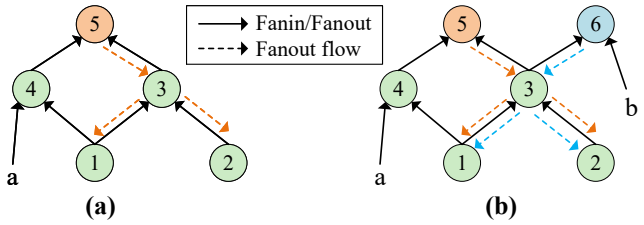


Figure 4: Illustration of fanout flow

3.2 Cut Enumeration

In traditional methods, such as IfMap, cut enumeration and selection are executed concurrently. For a node keeping at most C candidate cuts, the algorithm selects the top C cuts during the enumeration of C^2 cut merges from two fanins based on specific metrics (e.g., area flow). The highest-ranked cut is immediately assigned as the final cut, while the remaining $C-1$ cuts only assist the fanouts to enumerate their cuts.

However, in DiaMap, this phase is dedicated exclusively to enumerate C candidate cuts per node, and the final cut selection is deferred to subsequent procedures. Consequently, DiaMap avoids premature greedy selection in favor of maximizing candidate diversity. Three strategies are deployed in the exploration stage and two in the exploitation stage. As our primary objective is area optimization, these strategies must carefully balance area optimization with diversity. Three strategies at exploration stage are detailed below:

3.2.1 Direct Area Flow. As candidate cuts must be generated before any mapping results are established, this approach utilizes the area flow described in Eq. (2). For this equation, $R(n)$ represents the fanout count for each node within the AIG. Intuitively, the area flow computation operates on the premise that nodes with a larger number of direct fanouts will have a higher probability of being selected as cut leaves, their area costs may then be distributed among multiple cuts, effectively reducing their individual area contributions. The total cost of a cut is defined as the sum of its leaves' costs plus the intrinsic cost of the node itself.

3.2.2 Fanout-Aware Area Flow. Although direct area flow is ubiquitous in contemporary mapping algorithms, it only accounts for area propagation from fanins to direct fanouts, neglecting the influence of transitive fanouts. To address this limitation, we propose the *fanout-aware area flow*.

As illustrated in Figure 4, the likelihood of Node 1 being selected as a cut leaf differs between the two scenarios, despite having the same direct fanout count. This discrepancy arises from the variations in the fanout count of its fanout node, namely Node 3. Specifically, Node 5 not only can select Node 3 as a cut leaf, but also can expand the cut to include Node 1 and Node 2 (e.g., from {3, 4} to {1, 2, 4}), provided that the total number of leaves does not exceed the limit k. A similar observation holds for Node 6. Consequently, nodes with larger transitive fanout counts tend to have a higher probability of being selected as cut leaves. However, due to the cut size constraint k, this probability diminishes as the distance between node and the transitive fanout increases.

To capture this behavior, we propagate the *fanout flow* in reverse topological order as follows:

$$FF(n) = \sum_{v \in FO(n)} \alpha \cdot (R(v) + FF(v)) \tag{4}$$

Here, $FO(n)$ denotes the direct fanouts of n. $FF(n) = 0$ for POs. $0 \leq \alpha \leq 1$ is a weighting parameter that estimates the probability that a cut containing leaf v will be expanded to include node n. Incorporating this, the Fanout-Aware Area Flow of cut X is defined as:

$$AF_{fanout}(X) = \sum_{n \in X} \frac{AF_{fanout}(n)}{(1-\alpha) \cdot (R(n) + FF(n))} + 1 \tag{5}$$

Here, $\alpha = 0$ for PIs, as PIs have no fanins to be expanded.

3.2.3 Diversity-Aware Area Flow. As the subsequent differentiable cut selection phase is capable of searching a larger solution space, relying solely on the previous two strategies would yield overly homogeneous cuts, as they consistently favor nodes with higher fanout count and fanout flow. To mitigate this, we introduce a strategy that prioritizes cut diversity while maintaining a competitive area flow. The diversity score of a cut X for node n is calculated as:

$$DS(X, n) = \frac{|\{X\} \setminus CN(n)|}{|\{X\}|} \tag{6}$$

Here, $\{X\}$ denotes the set of leaves in cut X, and $CN(n)$ denotes the set of nodes from the cuts that have already been selected as candidate cuts for node n. The intuition is straightforward: a higher proportion of previously unseen leaves gives higher cut diversity. The *diversity-aware area flow* of cut X is then formulated as:

$$AF_{diversity}(X) = (1 - \beta \cdot DS(X, n)) \cdot AF(X) \tag{7}$$

Here, $0 \leq \beta \leq 1$ is a tunable parameter that dictates the influence of diversity on the final evaluation score.

These strategies are applied sequentially to assemble the candidate cuts. Specifically, assuming a node can at most have C candidate cuts, we select the first $\frac{1}{3}C$ cuts with the smallest direct area flow. The next $\frac{1}{3}C$ cuts are selected with the smallest fanout-aware area flow, and the remaining are chosen based on the diversity-aware area flow.

During the exploitation stage, the algorithm can leverage the mapping result obtained from the first stage. As a good optimization direction has been identified in the first stage, the algorithm can afford to be more greedy. Furthermore, cut expansion in the mapping result is significantly reduced, as

the process is guided by LUT levels rather than AIG node levels, eliminating the need to consider fanout flow. At this stage, we employ only the LUT-based direct area flow with Eq. (2) (selecting $\frac{2}{3}C$ of the cuts) and the diversity-aware area flow with Eq. (7) (selecting the remaining $\frac{1}{3}C$). Note that $R(n)$ in Eq. (2) denotes the fanout count derived from the LUT mapping obtained in the first stage, rather than from the AIG, since the objective is to refine the existing mapping. For unmapped nodes, $R(n)$ is set to 0.5.

3.3 Differentiable Cut Selection

Traditional cut selection relies on discrete, sequential heuristics, which are prone to be trapped in local optima. To enable scalable and effective search within the cut candidate space, we relax the discrete search space to become continuous and develop an differentiable solver accelerated by deep learning toolkits on GPUs [20].

3.3.1 Continuous Relaxation. We relax the categorical choice of cut candidates to selection probabilities. Let C_v denote the set of enumerated candidate cuts for node v, and let $P_{v,k} \in [0, 1]$ represent the probability of selecting the k-th cut for node v. To transform the constrained optimization problem ($\sum_k P_{v,k} = 1$) into an unconstrained one, we introduce trainable logits $w_{v,k} \in \mathbb{R}$. While a standard softmax function can map these logits to probabilities, its deterministic nature can inadvertently lead to local optima, especially under poor initializations [15]. Therefore, we employ the `gumbel_softmax` function [9] to introduce stochasticity and facilitate gradient backpropagation:

$$P_{v,k} = \frac{\exp((w_{v,k} + g_{v,k})/\tau)}{\sum_{j=1}^{|C_v|} \exp((w_{v,j} + g_{v,j})/\tau)} \tag{8}$$

Here, g denotes noise sampled from a standard gumbel distribution, and τ is the temperature parameter. As τ is gradually annealed toward zero during training, the distribution $P_{v,k}$ becomes increasingly peaked, approaching a discrete one-hot categorical distribution (i.e., a distribution in which a single entry takes value 1 while all others are 0), corresponding to the selection of exactly one cut. This process enables a smooth transition from continuous relaxation to discrete decision-making, thereby bridging the gap between gradient-based optimization and combinatorial mapping solutions.

3.3.2 Differentiable Area Evaluation. To optimize the cut probabilities $P_{v,k}$ via gradient descent, we formulate a continuous estimation of the total mapped area. Let $Y_v \in [0, 1]$ denote the mapped probability that node v is mapped as the root of an LUT in the final netlist. For POs, we set $Y_v = 1$. For any internal node u, whether it is mapped is determined by whether it is selected as a leaf in at least one final cut of mapped nodes.

Now, we aim to compute the probability that node u is selected as a leaf of a mapped node v. Since node v may have multiple candidate cuts and exactly one cut must be selected, these events are mutually exclusive. Therefore, the probability that the selected cut of v covers node u is given by the sum of the probabilities of all cuts that include u, rather than treating them as independent events.

Consequently, we define the demand $d_{v\to u}$ from node v to leaf u as the joint probability that v is mapped and the probability that its chosen cut covers u:

$$d_{v\to u} = Y_v \sum_{k:u\in C_{v,k}} P_{v,k} \tag{9}$$

Assuming that the demands from different fanout nodes are mutually independent, the mapped probability of node u can be expressed based on the demands from all its potential LUT fanout nodes. Let $\mathrm{PFO}(u)$ denote the set of nodes that have at least one candidate cut in which u appears as a leaf. Under this assumption, the mapped probability Y_u is given by the complement of the probability that none of its fanouts select it, i.e.,

$$Y_u = 1 - \prod_{v\in \mathrm{PFO}(u)} (1 - d_{v\to u}) \tag{10}$$

The total expected area of the mapped circuit is then formulated as the sum of the mapped probabilities of all nodes:

$$\mathrm{Loss}_{\mathrm{area}} = \sum_{v\in V} Y_v \tag{11}$$

This continuous formulation serves as our objective function, allowing gradients to be backpropagated from the global area cost down to the individual cut logits $w_{v,k}$.

To establish the theoretical soundness of our continuous relaxation, we prove that minimizing this differentiable loss function minimizes the actual area when the cut probabilities converge.

Theorem 1: Given a cut probability distribution **P** where $\sum_k P_{v,k} = 1$ for all $v \in V$, if the probabilities converge to a state such that $P_{v,k} \in \{0, 1\}$ for all v, k, the $\mathrm{Loss}_{\mathrm{area}}$ is equivalent to the true area of the mapped circuit, and the global minimum of $\mathrm{Loss}_{\mathrm{area}}$ corresponds to the global optimal area in current candidate cut search space.

Proof. Let $I_v \in \{0, 1\}$ denote the boolean indicator of whether a node v is mapped in the final deterministic mapping. For POs, $Y_v = 1 = I_v$. Assume inductively that for a fanout node v, $Y_v = I_v \in \{0, 1\}$. In a deterministic state, exactly one cut k^* is selected for v, meaning $P_{v,k^*} = 1$ and $P_{v,k} = 0$ for all $k \neq k^*$. Thus, the summation $\sum_{k:u\in C_{v,k}} P_{v,k}$ evaluates to 1 if $u \in C_{v,k^*}$, and 0 otherwise. Consequently, the demand $d_{v\to u} = Y_v \cdot \mathbb{I}(u \in C_{v,k^*}) \in \{0, 1\}$.

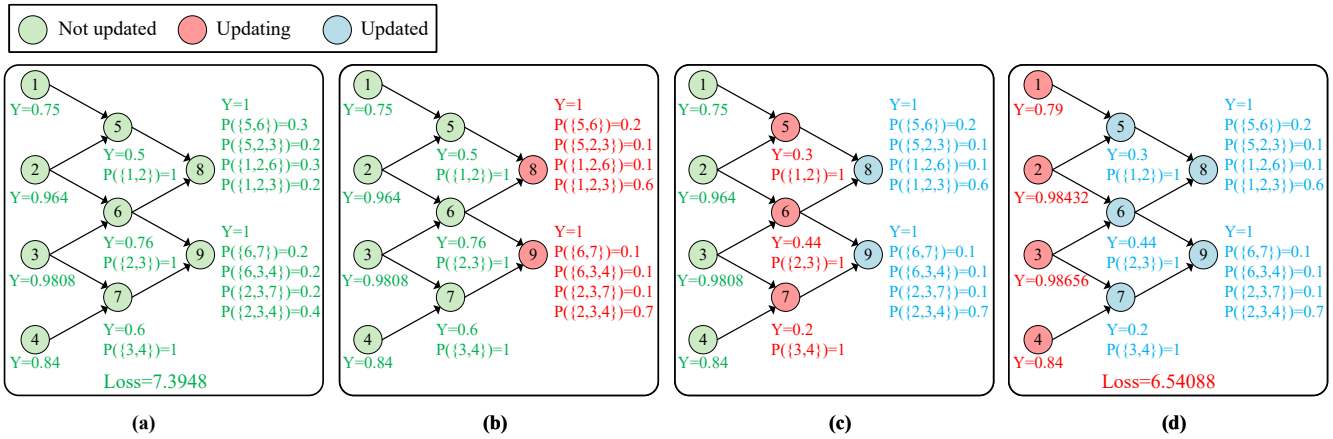


Figure 5: Illustration of propagation

Substituting this into Eq. (10), the product term $\prod_{v \in PFO(u)}(1 - d_{v \to u})$ evaluates to 0 if at least one potential fanout v demands u ($d_{v \to u} = 1$), and 1 if no fanouts demand u. Therefore, Y_u evaluates to exactly 1 or 0, which is consistent with the Boolean mapping indicator: $I_u = \bigvee_{v \in PFO(u)}(I_v \wedge \mathbb{I}(u \in C_{v,k^*}))$.

By mathematical induction, $Y_v = I_v$ for all $v \in V$. The loss function collapses to $\text{Loss}_{\text{area}} = \sum_{v \in V} I_v$, which is the actual area of the mapped circuit. Since our continuous function precisely interpolates the discrete area across the probability simplex, minimizing this loss is equivalent to finding the mapping with the minimum area. □

3.3.3 GPU Implementation and Acceleration. To fully leverage the massive parallelism of modern GPUs and to ensure numerical stability during gradient descent, we introduce two critical optimizations to our evaluation framework.

First, the cumulative product operation used to calculate the mapped probability Y_u in Eq. (10) poses a significant numerical challenge. In dense netlists, a node might have numerous potential fanouts. Multiplying a large number of probability terms $(1 - d_{v \to u})$, all bounded between 0 and 1, rapidly leads to floating-point underflow. Furthermore, computing the gradients of deep product chains is numerically unstable. To resolve this, we map the product into the log-space, transforming it into an exponential of a sum:

$$Y_u = 1 - \exp\left(\sum_{v \in PFO(u)} \log(1 - d_{v \to u})\right) \tag{12}$$

In practical implementation, to avoid the $\log(0)$ singularity when $d_{v \to u}$ approaches 1, the term inside the logarithm is lower-bounded by a small constant ϵ (e.g., 10^{-7}). This log-sum-exp transformation is mathematically equivalent to the original equation but prevents numerical underflow. More importantly, it converts the sequential multiplication into a parallel reduction sum, which is drastically more efficient to execute on GPU hardware.

Second, the calculation of the continuous area necessitates a reverse topological traversal of the AIG, as whether a node is mapped strictly depends on the demands from its potential fanouts. Processing this sequentially node-by-node would completely stall the GPU. To utilize the inherent parallelism of the circuit, we perform a levelized propagation.

Prior to the optimization loop, we group all nodes into topological levels based on their reverse logic depth from the POs. Nodes residing within the same topological level have no structural dependencies on one another. As illustrated in Figure 5, our GPU solver processes the circuit level by level. During the forward pass, the demands from all nodes in level L are computed and aggregated to update the mapped probabilities of nodes in level $L - 1$ as a single, highly parallelized tensor operation. The backward pass mirrors this levelized structure, propagating gradients concurrently across all edges between adjacent levels.

3.4 Top-p Reselection

The results of differentiable cut selection still leave room for improvement, as the differentiable optimizer cannot guarantee global optimality. To further enhance solution quality, we employ an incremental exact area change metric to re-evaluate the top-p candidate cuts and refine the selection.

Specifically, after differentiable cut selection, each cut is assigned a probability. For each node, we first select the cut with the highest probability as the initial choice and get the mapped results in a reverse topological order. We then travel the mapped node in topological order and re-evaluate the candidate cuts with top-p highest probabilities for each node

Table 1: Comparison with the area-oriented mapper in ABC

Benchmarks	AIG Statistics		ABC *if -a*			DiaMap (Ours)					
	#Nodes	Levels	#LUTs	Levels	Time (s)	#LUTs	Impr.(%)	Levels	Impr.(%)	Time (s)	Speedup
adder_10xd	1044480	255	254976	121	16.45	202895	20.43%	72	40.50%	12.88	1.28
bar_10xd	3416064	12	524288	4	61.95	524288	0.00%	4	0.00%	34.78	1.78
div_8xd	14655232	4372	6109440	2117	432.77	4219059	30.94%	2059	2.74%	238.62	1.81
hyp_6xd	13717440	24801	3043776	8402	470.64	2652932	12.84%	5015	40.31%	316.97	1.48
log2_9xd	16414720	444	3885056	160	649.83	3537313	8.95%	133	16.88%	270.76	2.40
max_10xd	2933760	287	737280	117	100.28	737280	0.00%	128	-9.40%	41.10	2.44
multiplier_9xd	13855744	274	2908160	126	461.53	2531982	12.94%	114	9.52%	217.02	2.13
sin_10xd	5545984	225	1484800	75	208.38	1332615	10.25%	65	13.33%	85.14	2.45
sqrt_9xd	12604416	5058	4139520	3955	428.36	2564238	38.05%	1479	62.60%	209.32	2.05
square_10xd	18927616	250	4089856	122	585.46	3487470	14.73%	100	18.03%	273.80	2.14
average	-	-	-	-	-	-	**14.91%**	-	**19.45%**	-	**2.00**
arbiter_10xd	12123136	87	2661376	19	395.80	2811412	-5.64%	20	-5.26%	159.42	2.48
cavlc_10xd	709632	16	118784	6	11.71	118784	0.00%	6	0.00%	7.47	1.57
ctrl_10xd	178176	10	29696	2	2.97	29696	0.00%	2	0.00%	2.92	1.02
dec_10xd	311296	3	293888	2	5.75	275453	6.27%	2	0.00%	5.46	1.05
int2float_10xd	266240	16	48128	5	3.62	48128	0.00%	5	0.00%	3.41	1.06
i2c_10xd	1374208	20	361472	7	19.40	366062	-1.27%	6	14.29%	15.80	1.23
mem_ctrl_8xd	11990016	114	2979328	53	283.13	2964633	0.49%	49	7.55%	150.55	1.88
priority_10xd	1001472	250	272384	105	16.11	222485	18.32%	42	60.00%	13.13	1.23
router_10xd	263168	54	74752	17	5.86	74336	0.56%	14	17.65%	4.04	1.45
voter_10xd	14088192	70	2836480	37	450.78	2227645	21.46%	23	37.84%	196.71	2.29
average	-	-	-	-	-	-	**4.02%**	-	**13.21%**	-	**1.53**

"_*n*xd" means that the benchmark is generated by enlarging the original one using ABC *double n* times.

to determine whether replacing the current selection can further reduce the area.

The evaluation metric is defined in Eq. (3). Interested readers are referred to [21] for more details. The parameter p controls the trade-off between computational efficiency and optimization quality.

4 Experimental Results

To evaluate the performance of the proposed differentiable area-oriented LUT mapping algorithm, we conduct experiments on 20 public benchmark circuits and compare the results with the IfMap in ABC [3].

4.1 Experimental Protocol

We implement DiaMap using PyTorch. All experiments are conducted on a server running Ubuntu 18.04 with an Intel Xeon Silver 4114 CPU, 256 GB memory, and an NVIDIA GeForce RTX 3090 GPU with 24 GB DRAM.

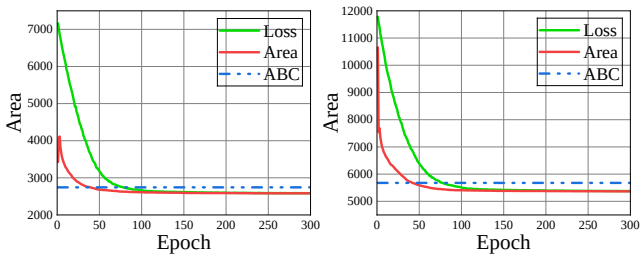
The benchmarks are selected from the EPFL benchmark suite [1], including both arithmetic and random control circuits. To evaluate scalability and demonstrate GPU parallelism, we enlarge the designs by repeatedly applying the ABC command *double*, a standard technique used in prior works [13, 27].

For comparison, we use the command *if -a* in ABC to perform area-oriented mapping. The maximum cut size is set to $k = 6$, and the number of priority cuts per node is $C = 8$, consistent with ABC. In DiaMap, the fanout flow transfer coefficient α and diversity coefficient β are set to 0.7 and 0.05, respectively. The reselection number p is set to 2.

We use the Adam optimizer with an initial learning rate of 0.1. The weights w are initialized uniformly. Both the exploration and exploitation stages run for 300 epochs. The initial temperature τ is set to 1 and is reduced by a factor of 0.9 every 100 epochs. All results are evaluated using ABC and verified by equivalence checking.

4.2 Computational Results

4.2.1 Comparison with IfMap. The comparison between DiaMap and IfMap is presented in Table 1. Here, "#Nodes" denotes the number of nodes in the input AIGs, and "#LUTs" represents the number of LUTs in the mapped circuits. "Levels" indicates the delay of the AIGs and the mapped circuits.



(a) voter **(b) multiplier**

Figure 6: Evolution of loss and area

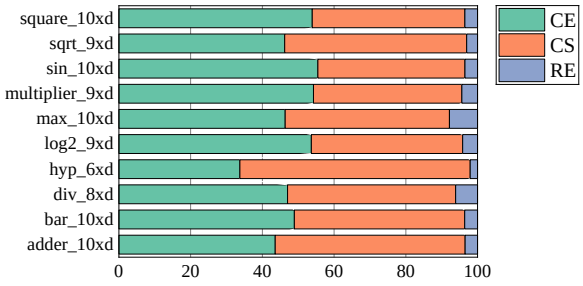


Figure 7: Runtime breakdown of DiaMap

"Time(s)" reports the execution time which does not include the file I/O time.

One can observe that DiaMap achieves better or comparable area for almost all circuits, while also providing smaller delay on average compared to IfMap. For arithmetic cases, DiaMap reduces area and delay by 14.91% and 19.45%, respectively. In particular, the area reduction exceeds 30% for cases "sqrt_9xd" and "div_8xd". All arithmetic cases achieve improved area, except cases "bar_10xd" and "max_10xd", where the area remains unchanged.

For random control cases, DiaMap still achieves average reductions of 4.02% in area and 13.21% in delay. The relatively smaller improvement is likely due to the simpler circuit structures and smaller search space, as many cases result in identical area and delay. In a few cases (e.g., "arbiter_10xd" and "i2c_10xd"), DiaMap produces slightly larger area, although the difference is marginal.

In terms of runtime, DiaMap achieves average speedups of 2× and 1.53× for arithmetic and random control cases, respectively. This improvement is attributed to the differentiable mapping framework, which can effectively exploits GPU parallelism. And the speedup will become larger as the circuit size increases.

4.2.2 Convergence Analysis. Figure 6 illustrates the evolution of the loss and area for the cases "voter" and "multiplier" during the exploration stage of differentiable cut selection. Here, "Loss" corresponds to Eq. (11), and "Area" denotes the number of LUTs at each epoch.

The results show that the loss and area follow a consistent decreasing trend. Initially, there is a noticeable gap between the loss and the actual area. However, this gap gradually diminishes as the optimization proceeds. This behavior can be attributed to two factors. First, the optimizer progressively identifies better optimization directions. Second, as the temperature τ in Eq. (8) decreases, the selection probabilities become increasingly discrete (i.e., closer to 0 or 1 rather than intermediate values such as 0.5), causing the loss to better approximate the actual area, as supported by Theorem 1.

Furthermore, both loss and area decrease steadily throughout the optimization process, demonstrating the effectiveness of the proposed loss design.

4.2.3 Runtime Analysis. Figure 7 shows the runtime breakdown of cut enumeration (CE), differentiable cut selection (CS), and reselection (RE) for arithmetic cases. On average, these three phases account for 48%, 48%, and 4% of the total runtime, respectively. The primary bottlenecks lie in cut enumeration and cut selection. Notably, the current cut enumeration phase is executed on the CPU and can be further accelerated using parallelization techniques such as those proposed in [13].

We also observe that, for the case "hyp_6xd", cut selection dominates the runtime, accounting for nearly 65%. This is because differentiable cut selection is performed in a level-by-level manner, where parallelism is limited to nodes within the same level. Since this circuit has a large number of AIG levels, the available parallelism is restricted, leading to reduced GPU utilization.

5 Conclusion

In this paper, we propose a differentiable area-oriented LUT mapping algorithm that enables efficient exploration of the global search space. Specifically, we introduce three strategies for cut enumeration that consider both fanout flow and cut diversity. We then develop a differentiable cut selection framework that can be efficiently accelerated using GPU-based deep learning toolkits. Finally, we propose a top-p reselection method to further reduce the mapped area. Experimental results on arithmetic benchmarks show that our approach achieves average reductions of 14.91% in area and 19.45% in delay compared to the area-oriented mapper in ABC. In addition, DiaMap provides a 2× runtime speedup.

Acknowledgments

The work described in this paper was partially supported by a grant from the Research Grants Council of the Hong Kong Special Administrative Region, China (Project Reference Number: T46-415/25-R).

References

[1] Luca Amarú, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. 2015. The EPFL combinational benchmark suite. In *Proceedings of the 24th International Workshop on Logic & Synthesis (IWLS)*.

[2] Luca Amarú, Vinicius Possani, Eleonora Testa, Felipe Marranghello, Christopher Casares, Jiong Luo, Patrick Vuillod, Alan Mishchenko, and Giovanni De Micheli. 2021. LUT-based optimization for ASIC design flow. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 871–876.

[3] Robert Brayton and Alan Mishchenko. 2010. ABC: An academic industrial-strength verification tool. In *International Conference on Computer Aided Verification*. Springer, 24–40.

[4] Deming Chen and Jason Cong. 2004. DAOmap: A depth-optimal area optimization mapping algorithm for FPGA designs. In *IEEE/ACM International Conference on Computer Aided Design, 2004. ICCAD-2004.* IEEE, 752–759.

[5] Jason Cong and Yuzheng Ding. 1994. FlowMap: An optimal technology mapping algorithm for delay optimization in lookup-table based FPGA designs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 13, 1 (1994), 1–12.

[6] Jason Cong and Yean-Yow Hwang. 1995. Simultaneous depth and area minimization in LUT-based FPGA mapping. In *Proceedings of the 1995 ACM third international symposium on Field-programmable gate arrays*. 68–74.

[7] Yufan Du, Zizheng Guo, Yibo Lin, Runsheng Wang, and Ru Huang. 2024. Fusion of global placement and gate sizing with differentiable optimization. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.

[8] Zizheng Guo, Zuodong Zhang, Wuxi Li, Tsung-Wei Huang, Xizhe Shi, Yufan Du, Yibo Lin, Runsheng Wang, and Ru Huang. 2024. HeteroExcept: A CPU-GPU heterogeneous algorithm to accelerate exception-aware static timing analysis. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.

[9] Eric Jang, Shixiang Gu, and Ben Poole. 2016. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144* (2016).

[10] Wei Li, Rongjian Liang, Anthony Agnesina, Haoyu Yang, Chia-Tung Ho, Anand Rajaram, and Haoxing Ren. 2024. Dgr: Differentiable global router. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.

[11] Yibo Lin, Shounak Dhar, Wuxi Li, Haoxing Ren, Brucek Khailany, and David Z Pan. 2019. Dreamplace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6.

[12] Siting Liu, Yuan Pu, Peiyu Liao, Hongzhong Wu, Rui Zhang, Zhitang Chen, Wenlong Lv, Yibo Lin, and Bei Yu. 2022. Fastgr: Global routing on cpu–gpu with heterogeneous task graph scheduler. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 7 (2022), 2317–2330.

[13] Tianji Liu, Yang Sun, Lei Chen, Xing Li, Mingxuan Yuan, and Evangeline FY Young. 2024. A unified parallel framework for lut mapping and logic optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 1 (2024), 214–226.

[14] Qiang Ma and Evangeline FY Young. 2010. Multivoltage floorplan design. *IEEE transactions on computer-aided design of integrated circuits and systems* 29, 4 (2010), 607–617.

[15] Chris J Maddison, Andriy Mnih, and Yee Whye Teh. 2016. The concrete distribution: A continuous relaxation of discrete random variables. *arXiv preprint arXiv:1611.00712* (2016).

[16] Wai-Kei Mak and Evangeline FY Young. 2002. Temporal logic replication for dynamically reconfigurable FPGA partitioning. In *Proceedings of the 2002 international symposium on Physical design*. 190–195.

[17] Valavan Manohararajah, Stephen Dean Brown, and Zvonko G Vranesic. 2006. Heuristics for area minimization in LUT-based FPGA technology mapping. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25, 11 (2006), 2331–2340.

[18] Alan Mishchenko, Satrajit Chatterjee, and Robert Brayton. 2006. Improvements to technology mapping for LUT-based FPGAs. In *Proceedings of the 2006 ACM/SIGDA 14th international symposium on Field programmable gate arrays*. 41–49.

[19] Walter Lau Neto, Luca Amarú, Vinicius Possani, Patrick Vuillod, Jiong Luo, Alan Mishchenko, and Pierre-Emmanuel Gaillardon. 2022. Improving LUT-based optimization for ASICs. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 421–426.

[20] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).

[21] Heinz Riener, Winston Haaswijk, Alan Mishchenko, Giovanni De Micheli, and Mathias Soeken. 2019. On-the-fly and DAG-aware: Rewriting Boolean networks with exact synthesis. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1649–1654.

[22] Hsin-Ying Tsai, Chung-Kai Wu, Chih-Cheng Hsu, and Jie-Hong R Jiang. 2025. GradMap: A Gradient-Descent Approach to Simultaneous Technology Mapping, Buffer Insertion, and Gate Sizing. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.

[23] Peiyu Wang, Anqi Lu, Xing Li, Junjie Ye, Lei Chen, Mingxuan Yuan, Jianye Hao, and Junchi Yan. 2023. Easymap: Improving technology mapping via exploration-enhanced heuristics and adaptive sequencing. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 01–09.

[24] Douglas Brent West et al. 2001. *Introduction to graph theory*. Vol. 2. Prentice hall Upper Saddle River.

[25] Liang Xiao, Shiju Lin, Jinwei Liu, Qinkai Duan, Tsung-Yi Ho, and Evangeline FY Young. 2025. InstantGR: Scalable GPU parallelization for 3-d global routing. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).

[26] Linfu Xiao, Subarna Sinha, Jingyu Xu, and Evangeline FY Young. 2010. Fixed-outline thermal-aware 3D floorplanning. In *2010 15th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 561–567.

[27] Zhenxuan Xie, Lixin Liu, Tianji Liu, and Evangeline FY Young. 2025. ExactMap: Enhancing Delay Optimization in Parallel ASIC Technology Mapping. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.

[28] Wenlong Yang, Lingli Wang, and Alan Mishchenko. 2012. Lazy man's logic synthesis. In *Proceedings of the International Conference on Computer-Aided Design*. 597–604.